

Professional OTLP - Training

Facilitator: Ramkumar J Dhamodharan

<https://www.linkedin.com/in/ramkumar-j-d-57423611/>

Training Agenda

- OTLP - Introduction
- Monitoring & Observability - Concepts
- Open Source Stack for M&O | OTLP
 - Prometheus
 - Grafana
 - Loki
 - Tempo
 - Jaeger
- Instrumentation
- Zero Code Instrumentation
- Manual Instrumentation
- Resource & Semantic Conventions
- OTLP Collector
- Processing & Data Transformation
- Various Connectors
- Security & Governance
- Dashboards & Alerts
- AI in Monitoring & Observability

Monitoring and Observability

- Monitoring answers questions you already knew to ask.
 - It's very good at known problems
- Observability is different
 - It means you can ask a brand-new question about your system, from the outside, without shipping a new code.
 - It's very good at Unknown Questions | scenarios
- Observability is part of your system - property of the system
- Producing good telemetry is exactly what OpenTelemetry is all about

Monitoring = Questions you knew to ask

Observability = New Questions | No New Code

Monitoring -> That's something wrong

Observability - WHY, even for ONE request

Differences

Monitoring

Known Problems

Dashboards | Alerts

CPU > 80%

Monoliths

Observability

New Problems

Ask from the outside

Why was this order SLOW?

Microservices

Telemetry - The Three Signals

- It comes with three main forms
 - Often they're called as SIGNALS
 - Metrics
 - Logs
 - Traces
- Metrics - are numbers that are measured over time
 - Mostly they're stored in TSDB (Time Series Database)
 - Prometheus
- Logs - are timestamped records of individual events
 - Loki
- Traces - It shows the journey of one request across all the services it touches
 - A trace is made of spans
 - A span is ONE UNIT of WORK
 - Tempo | Jaeger
- Profiles & Events

What to Measure?

- Golden Signals
- RED
- USE

- Google SRE practices describe the four Golden Signals
 - Latency
 - Traffic
 - Errors
 - Saturation

- For services that handle requests, like an Order API, people use RED
 - R - Rate
 - E - Errors
 - D - Duration

- For resources - A Host, A Queue, A Connection Pool - USE
 - U - Utilization
 - S - Saturation
 - E - Errors

An easy way to remember it

- RED - How your users experience a service
- USE - Whether the machine underneath is STRUGGLING or OVERUTILIZED?

Reliability Targets

- SL*
 - SLI
 - SLO
 - SLA
- Error Targets
- SLI - Service Level Indicator
 - It's a measurement
 - It's usually a ratio
 - For example, the share of Order Requests that succeed in under 500ms
- SLO - Service Level Objective
 - It's the target for that measurement over a time window
 - SLO owned by Engineering | Product together
- SLA - Service Level Agreement
 - It's a contract with the customer, with consequences if you miss it - Usually money back or service credits
 - Owned by Business and Legal Teams
- SLO is more stricter than SLA
- Error Budgets & Targets
- If your target is 99.5% - Then 0.5% of requests are allowed to fail - Allowance is your budget
- Burn Rate
 - How fast you're spending the budget
 - A burn rate of ONE means you will use exactly the whole budget by the end of the window
 - A burn rate of TEN means you will run out in a tenth of the time.

OTel defines NO SLOs OOB - It feeds clean metrics

Bad instrumentation means bad SLOs.

Telemetry costs **MONEY**

- Three quick points about the cost
 - Cardinality
 - Sampling
 - Logs
- Cardinality - Every unique combination of a metric name and its label values is a SEPARATE TIME SERIES
- Sampling - Keeping every single trace is EXPENSIVE
 - Head Sampling
 - Tail Sampling
- Logs - Most expensive signal
 - When your traces are GOOD and connected to your logs, teams can often log less

Why OpenTelemetry?

- Interoperability
- Compatibility
- Portability
- Lock-In & Expensive Solutions

- OpenTelemetry (OTEL) is a CNCF project
- It standardizes everything before the data reaches a back-end
 - The API you code against
 - The SDKs
 - The Wire Protocol called OTLP
 - The Collector
 - Even the Names of Attributes
- The Promise is
 - Instrument ONCE, SEND anywhere.
- Vendor Agent
 - Vendor Lock-In
- OTel is NOT a back-end; No OTel Dashboards

API + SDK = Instrumentation

OTLP = Wire Protocol

Collector = Process & Route

Any Back-Ends = Grafana | Prometheus | Datadog ...

**** OTEL = EVERYTHING BEFORE THE BACKEND ****

Open Source Tech Stack for Monitoring & Observability

- Prometheus
- Grafana
- Loki
- Tempo / Jaeger

They're the core open-source pillars of Modern Cloud Native Observability, designed to handle the "Three Pillars" of Telemetry - Metrics, Logs and Traces & unified through a Visualization Layer (Grafana)

Tool	Core Pillar	Primary Function	Storage Model	Query Language
Prometheus	Metrics	Numerical Time Series Monitoring and Alerting	Built-in TSDB (Time Series Database)	PromQL
Grafana	Visualization	Unified Dashboards, Analytics & Alerting across 100+ connectors	Stateless Front-End (Connects to External Back-ends)	UI / Dashboard (No Query Language Supported)
Jaeger	Traces	Distributed Request Tracing & RCA	Pluggable (Elastic Search, Cassandra, ClickHouse)	Jaeger UI / Query API
Loki	Logs	Log Aggregation via Indexed Metadata and Labels	Object Storage - S3, Azure Blob Storage, GCS	LogQL
Tempo	Traces	High Volume, Cost Effective, Distributed Tracing	Object Storage (S3, Azure Blob Storage, GCS)	TraceQL Grafana Dashboards

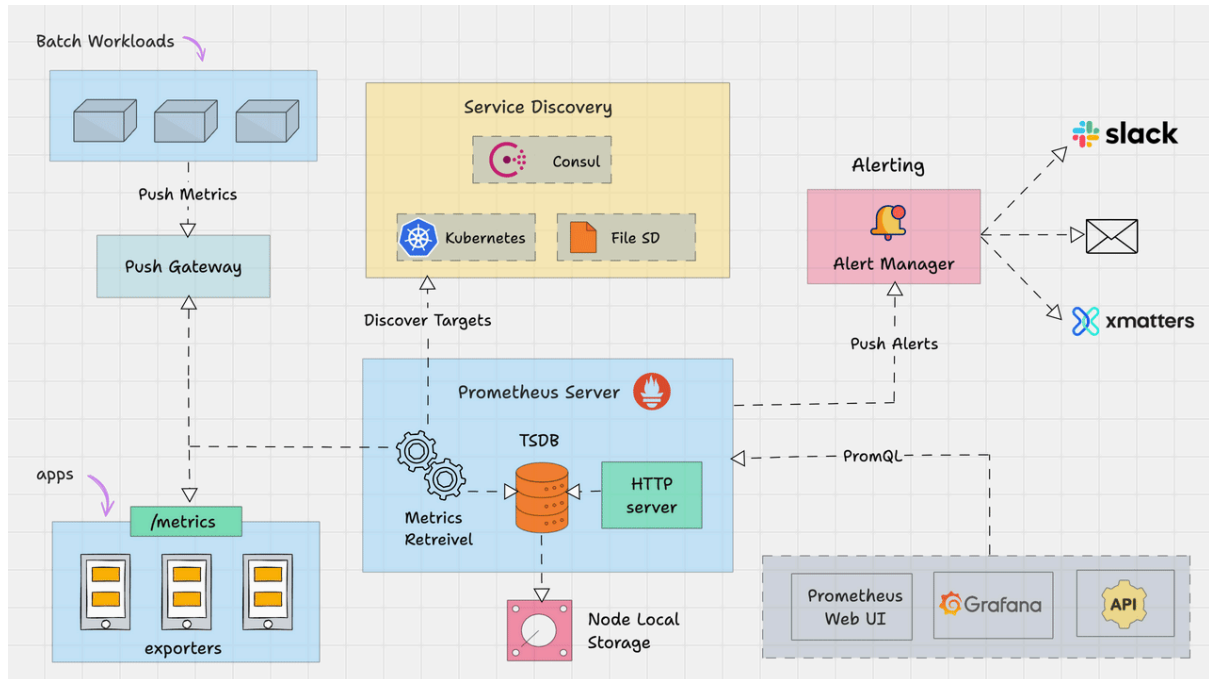
Prometheus

- It's an Open Source Monitoring & Alerting tool
- It follows an unique architecture designed to collect, store and process metrics from various sources in a scalable and efficient manner

Architecture

- Prometheus Server
 - There are three parts in it
 - Data Retrieval Worker
 - This worker is responsible for collecting the metrics from the targets, so it will send the HTTP request out to the target & collect the metrics. Once it's collected, the metrics will be stored in the TSDB
 - TSDB
 - This database is just where we store all the metrics and the database in Prometheus plays a crucial role in the efficient storage, retrieval and management of time-series metrics
 - HTTP Server
 - It's just there to allow us to retrieve the data stored in the TSDB, and then we want to take a look at that data and visualize.
- Exporters
 - Exporters are intermediaries between the target system and Prometheus. They extract metrics from the target system, transform them into the Prometheus Data format and expose an HTTP endpoint for the Prometheus server to scrape the metrics
 - Various Exporters available OOB
 - Windows
 - Node Exporters for Linux
 - MySQL | Postgres Exporters
- Client Libraries
 - It comes with client libraries that allow you to expose any application metrics you need Prometheus to work | track
- Push Gateway
 - While Prometheus primary follows a pull-based model, it also provides a component named "Push Gateway" for scenarios where metrics need to be pushed by SHORT-LIVED or BATCH JOBS that can NOT be scraped
- Alert Manager
 - It handles alerts generated by Prometheus, and allows for routing, grouping and processing of alerts, which can then be sent to various notification channels such as Email, Chat | Incident Management platforms

- Service Discovery
 - It's all about providing a list of targets to Prometheus so you do NOT have to hard code those target values explicitly.
 - Discover and Monitor Targets automatically without requiring manual configuration



Grafana

- A modular framework for TS Data Visualization
- A framework that enables Monitoring, Querying and Visualizing TS data from multiple sources
- Back-End and Front-End Application Architecture
- Front-End
 - UI interactions through the Web Application
 - Interactive Dashboards Creation, Customization and Data Source Management
 - Communication with Back-End REST APIs
- Back-End
 - Built with Go
 - Handling Authentication, Data Source Management, Querying and Alerting
 - Communicates with External Data sources, Process Queries and Sends results to Front-End
 - Alerting Engine - Evaluates conditions, Triggers Alerts and Sends Notifications
- Data Source Plugins
- Storage - Stores dashboard metadata, user preferences and alert rules in databases (SQLite, Postgres, MySQL)
- Authentication System
 - OAuth
 - LDAP
 - Basic Authentication
 - RBAC (Role Based Access Control)
- Grafana Cloud - Managed service that simplifies deployment and scaling of Grafana, Prometheus and Loki in the cloud

Loki

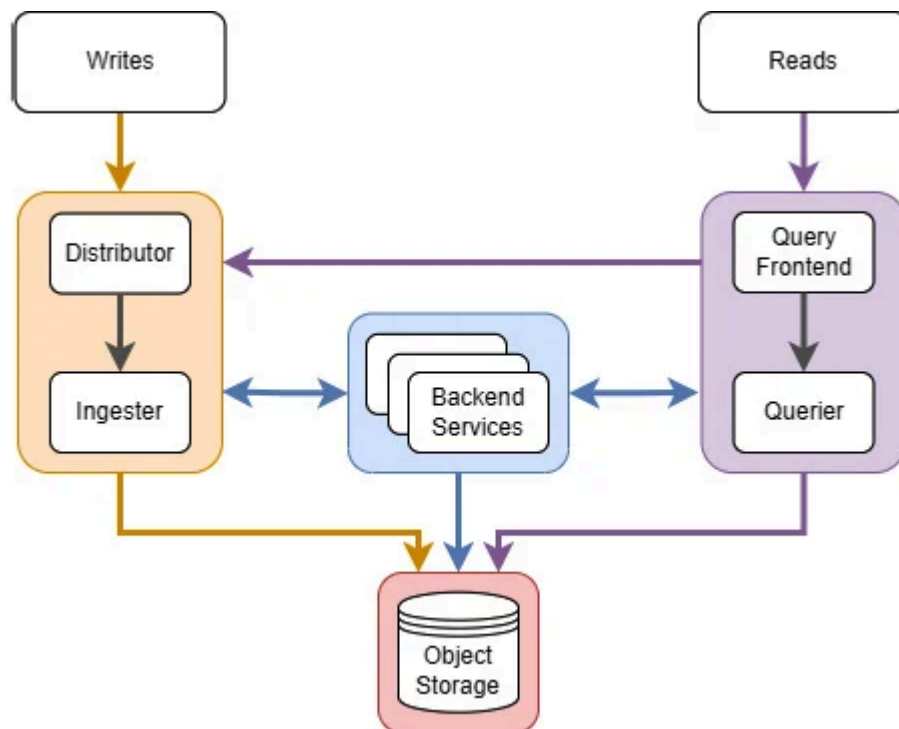
- It's horizontally scalable, highly available and multi-tenant log aggregation system
- It differs from Prometheus
 - By focusing on logs instead of Metrics
 - Collecting Logs via Push, instead of Pull
- It does NOT index the contents of the logs, but only INDEXES metadata about your logs as a set of labels for each log stream
- A log stream is a set of logs which share the same labels. Labels help Loki to find a log stream within your data storage, so having a quality set of labels is KEY to efficient query execution
- Log data is then compressed and stored in chunks in object store such as S3, GCS or file system
- A small index and highly compressed chunks simplify the operation and significantly reduce the cost of Log Aggregation and Log Processing



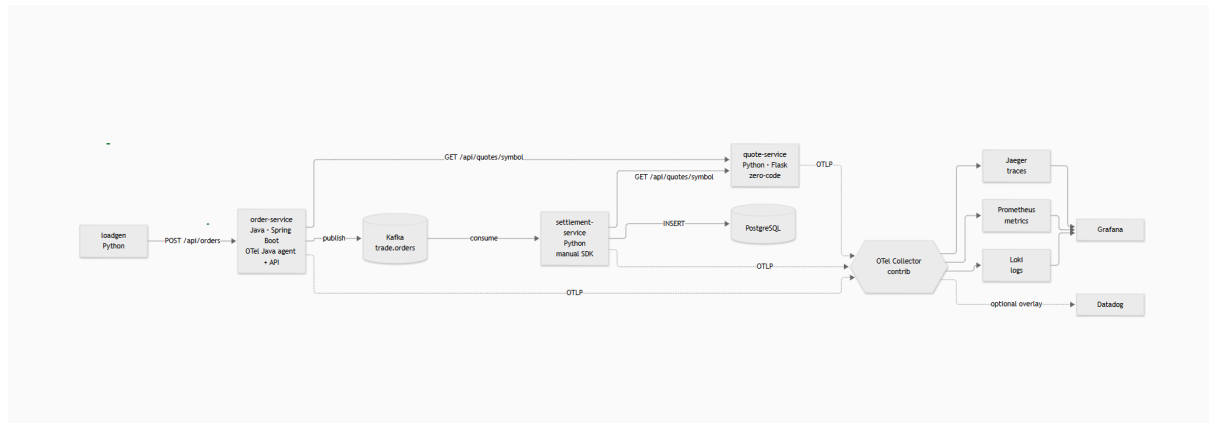
- 3 Components
 - Agent
 - An agent or client scrapes logs, turns the logs into streams by adding labels and pushes the streams to Lok through an HTTP API
 - Loki
 - The main server, responsible for ingesting and storing logs and processing queries
 - Client (Grafana / CLI / API)
 - For querying and displaying log data

Tempo

- It's an Open-source, easy-to-use and highly scalable distributed tracing back-end
- It lets you search for traces, generate metrics from spans and link your tracing data with logs and metrics
- It acts in two major capacities
 - Ingesting trace spans, sorting the span resources and attributes into columns in an Apache Parquet format, before sending them to Object Storage for long-term retention
 - Retrieving trace data from the storage, either by Specific Trace ID or by search parameters using TraceQL



Business Use Case - Application Architecture



Mermaid

flowchart LR

```

    LG[loadgen<br/>Python] -->|POST /api/orders|
    OS[order-service<br/>Java · Spring Boot<br/>OTel Java agent + API]
    OS -->|publish| K[(Kafka<br/>trade.orders)]
    K -->|consume| SS[settlement-service<br/>Python<br/>>manual SDK]
    SS -->|GET /api/quotes/symbol| QS[quote-service<br/>Python · Flask<br/>zero-code]
    SS -->|INSERT| PG[(PostgreSQL)]
    OS -. OTLP .-> C[OTel Collector<br/>contrib]
    QS -. OTLP .-> C
    SS -. OTLP .-> C
    C --> J[Jaeger<br/>traces]
    C --> P[Prometheus<br/>metrics]
    C --> L[Loki<br/>logs]
    C -. optional overlay .-> DD[Datadog]
    J --> G[Grafana]
    P --> G
    L --> G
  
```

Questions to be answered

- Nobody knows about what our telemetry actually contains
- Instrumenting every service by hand will take a year, and some job is flooding our traces
- We can't see the risk check, one metric is exploding in cost and our logs are NOT linked to traces
- Trades vanish between Order and Settlement
- The same service has three different names - and our dashboards don't JOIN them UP

Lab Environment Setup

> Open Cloud Lab with your credentials

> in the task bar -> terminal -> right click -> run as administrator

> cd lab-systems

> pwsh -version

(note: there are two dashes)

> pwsh .\installation-check.ps1 -Otlp

NOTE: if installation checks are FAILING (you can ignore SQL Server Management Studio tool check), you can then run the following command.

> pwsh .\install-lab-software-otlp.ps1

For reference, you can use the following GitHub repository

<https://github.com/iomegak12/lab-systems>

Get the Source Code from GitHub

Open the terminal window in Administrator Mode

> md c:\demonstrations

> cd c:\demonstrations

> git clone <https://github.com/iomegak12/otlp-demos-1>

> cd otlp-demos-1

> code .

> view -> terminal

> docker --version
(two dashes)

> docker compose build

> docker images

> docker compose pull

> docker images

> in vs code -> extensions

> search for Docker Compose Flow Visualizer

> install the extension

> double click docker-compose.yml and should be able to see the visual flow of all containers

> in vs code -> terminal

> docker compose up -d

> docker compose ps

> open docker desktop -> expand tradenova folder -> verify all containers are UP and RUNNING (10 containers)

> open the browser

> <http://localhost:3000> (Grafana)

- > <http://localhost:9090> (Prometheus)
- > <http://localhost:16686> (Jaeger UI)
- > in the VS Code terminal window ->
- > uv sync
- > .venv\scripts\activate
- > open .env file and update the basic to detailed for the setting DEBUG_VERBOSITY
- > docker compose up -d otel-collector
- > docker compose logs -f otel-collector

To stop following up the container logs, press CTRL+C

- > python --version
(two dashes - it should show it as 3.12)
- > python labs\lab-a\send_otlp.py --show
(two dashes)
- > in the jaeger ui, you should now be able to see the service lab-a-cli
- > search for traces
- > you should see a trace and multiple spans
- > open the prometheus
- > query lab_orders_placed_total
- > execute the query and change the view to graph mode
- > open grafana
- > explore -> choose the source loki
- > service_name = lab-a-cli
- > execute the query
- > include trace_id
- > open the log
- > click the button open the log in jaeger

> in vs code -> terminal -> click + to open the git bash

> paste the following

```
curl -s -X POST http://localhost:8080/api/orders \
-H 'Content-Type: application/json' \
-d '{"accountId":"ACC-10001","symbol":"TNVA","side":"BUY","quantity":10}'
```

> make sure that order is ACCEPTED

> in Jaeger -> Services List -> verify TradeNova settlement service is listed ... why only one?

> click find traces

> make sure that you see postgres DB insert ...

Zero Code Instrumentation

> open .env file and update the DEBUG_VERBOSITY to basic

> in VS Code -> Terminal, docker compose up -d otel-collector

> in VS Code -> Terminal, docker compose --profile load up -d loadgen
(two dashes)

> docker compose logs loadgen --tail 5
(two dashes)

Open the jaeger UI -> look at the settlement service trace, still only settlement service ... is not it?

> open the config\order-service.env

> you can see JAVA_TOOL_OPTIONS attaches the agent to the JVM

> change the `OTEL_JAVAAGENT_ENABLED` to `true`

> docker compose up -d order-service

> git bash -> docker compose logs order-service | grep -i opentelemetry

> you should be able to see open telemetry java agent is running with appropriate version

> open the file config\quote-service.env and change

`QUOTE_ZERO_CODE` to `true`

> if you want to understand how the changes are taken in effect, open the file `services\quote-service\start.sh` and see how the shell file is designed

> `docker compose up -d quote-service`

> `docker compose logs quote-service --tail 25`
(two dashes)

In the Jaeger UI -> Now, you should be able to see

Order Service, Quote Service and all are lined up

Search for `error=true` find traces

In Prometheus UI -> run the following queries

> `http_server_request_duration_seconds_count`

> `http_server_duration_milliseconds_count`

> `jvm_memory_used_bytes`

> Grafana -> Explore -> Choose Loki

> query `service_name = OrderService`

> Run query

> Select Trace Id and Show links in Log Lines -> Open in Jaeger

Remove the Noise with Configuration

Open `config/quote-service.env`

Add the following line

```
OTEL_PYTHON_FLASK_EXCLUDED_URLS=health
```

Open `config/order-service.env`

Change the setting from `false` to `true`

```
OTEL_INSTRUMENTATION_SPRING_SCHEDULING_ENABLED=true
```

> File -> Save All

> in the terminal window,

> docker compose up -d order-service quote-service

> wait for a minute

> in Jaeger UI, select Order Service -> set lookback to 5 minutes -> find traces

> now you should be able to see additional noisy endpoint (refreshSessionStatus)

> select Quote Service -> find traces -> health endpoint should NOT be appearing

Manual Instrumentation

How do you add a span for custom attributes or details?

Open

services\order-service\src\main\java\com\tradenova\order\RiskService.java file, and try to understand how to add custom spans

```
Span span = Span.current();
span.setAttribute("tradenova.risk.limit", ACCOUNT_LIMIT);
span.setAttribute("tradenova.risk.decision",
decision.name().toLowerCase(Locale.ROOT));
return decision;
```

How do you add a custom metric and histogram to Prometheus Metrics collection?

Open the file

services\order-service\src\main\java\com\tradenova\order\OrderController.java

In the constructor,

```

        Meter meter =
GlobalOpenTelemetry.getMeter("com.tradenova.order");
        this.ordersPlaced =
meter.counterBuilder("tradenova.orders.placed")
            .setDescription("Orders received by order-service, by
side and outcome")
            .setUnit("{order}")
            .build();

        this.orderValueHistogram =
meter.histogramBuilder("tradenova.order.value")
            .setDescription("Value of accepted orders")
            .setUnit("{USD}")
            .setExplicitBucketBoundariesAdvice(
                List.of(1_000.0, 5_000.0, 10_000.0, 25_000.0,
50_000.0, 100_000.0, 250_000.0))
            .build();

```

In the business logic of placeOrder(),

```

        ordersPlaced.add(1, Attributes.of(SIDE, side, OUTCOME,
"accepted"));
        orderValueHistogram.record(orderValue, Attributes.of(SIDE,
side));

```

In the terminal window,

```
> docker compose up -d --build order-service
```

> In Jaeger UI,

-> select Order Service, the span now should see additional attributes

> In Prometheus UI,

```

histogram_quantile(0.95, sum by (le)
(rate(tradenova_order_value_bucket[1m])))

```

```
sum by (outcome) (rate(tradenova_orders_placed_total[1m]))
```

Change the view to Graph -> execute

Context Propagation Techniques

How do you stitch disconnected traces across services and its communications, especially when a service communicates with another service asynchronously?

See the breakage

> In Jaeger UI,

> An OrderService trace stops at Kafka Publish

> Settlement Traces start at trade.orders.process Kafka Consumer without any parent details

> Open the settlement service which is NOT linked with Notify Customer Trace

How do you fix?

> terminal window -> paste the following in one line

```
docker compose exec kafka /opt/kafka/bin/kafka-console-consumer.sh
--bootstrap-server localhost:9092 --topic trade.orders --property
print.headers=true --max-messages 3
```

Find transparent and baggage attributes in the message header ...

Fix #1

Open the file services\settlement-service\[main.py](#)

Add the following imports

After line 17,

```
import contextvars
```

At line #25,

```
from opentelemetry import baggage, context, metrics, trace
from opentelemetry.propagate import extract, inject
```

At line #117,

Comment settle() method

And paste the following

```
carrier = {key: value.decode(
    "utf-8") for key, value in (message.headers() or []) if value
is not None}
token = context.attach(extract(carrier))
try:
    settle(message, order, conn)
finally:
    context.detach(token)
```

Save all

In the terminal window

```
> docker compose up -d --build settlement-service
```

Wait for a minute ...

In the Jaeger UI, choose Order Service -> POST /api/orders, now you should be able to see settlement service also attached together

Open the file
services\settlement-service\[main.py](#)

Line #76,

Right after headers={}, paste the following

```
inject(headers)
```

Line #164,

Comment notification_pool.submit() line, and paste the following

```
notification_pool.submit(contextvars.copy_context().run,  
notify_customer, order)
```

Line #142,

Add the following right after tradenova.order.symbol attribute has been set

```
channel = baggage.get_baggage("tradenova.channel")  
if channel is not None:  
    span.set_attribute("tradenova.channel", str(channel))
```

In the terminal window

```
> docker compose up -d --build settlement-service
```

Wait for a minute,

In Jaeger UI, select OrderService -> POST /api/orders

Select any trace

Now, you should be able to see the complete trace including Order Service -> Quote Service -> Settlement Service -> Notify Customer

In the Jaeger UI, Select Settlement Service -> search for attributes

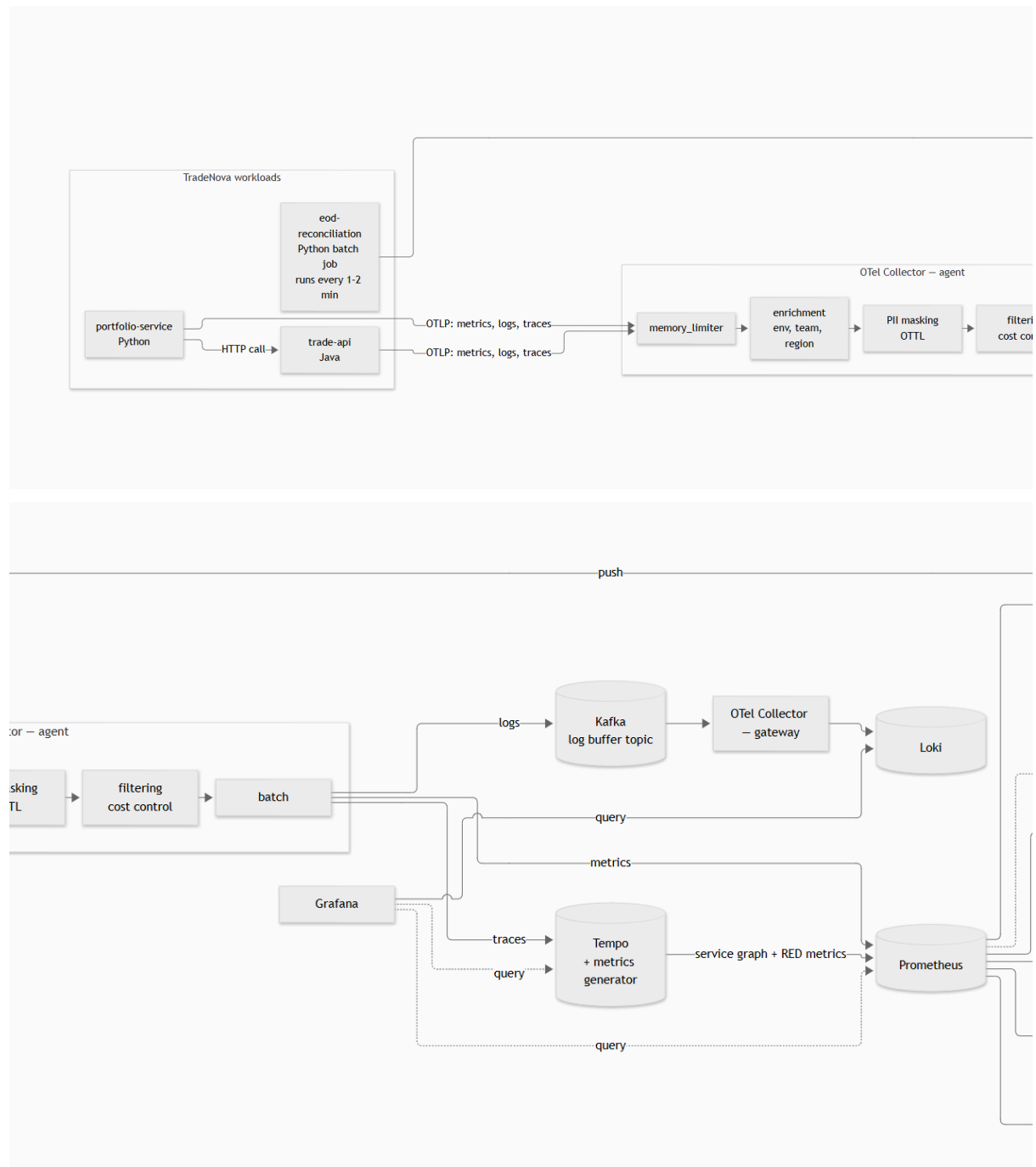
tradenova.channel=mobile

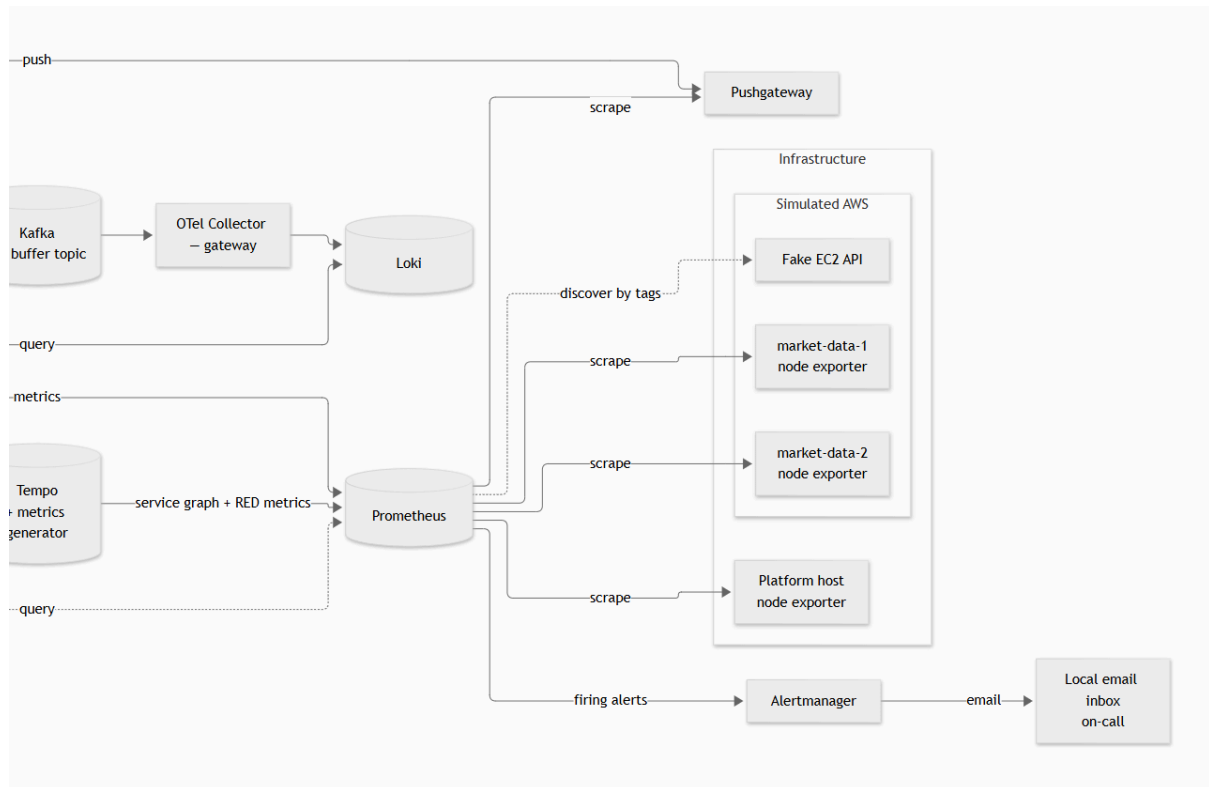
Find traces, now you're able to find the trace specific to business attributes

Day 2 Agenda

- Day 1 Summary
- OTel Collector Agents
- Receivers & Exporters
- Processors & Data Transformation
- RED / USE Patterns
- Enterprise Business Use Case - 2
 - Common Enterprise Challenges in M&O
- Demonstrations & Practices

Today's Theme: From Pilot to Platform





flowchart LR

```

subgraph APPS["TradeNova workloads"]
    PS["portfolio-service<br/>Python"]
    TA["trade-api<br/>Java"]
    EOD["eod-reconciliation<br/>Python batch job<br/>runs every 1–2 min"]
end

```

```

PS -->|HTTP call| TA

```

```

subgraph AGENT["OTel Collector — agent"]
    direction TB
    P1["memory_limiter"] --> P2["enrichment<br/>env, team, region"]
    P2 --> P3["PII masking<br/>OTTL"]
    P3 --> P4["filtering<br/>cost control"]
    P4 --> P5["batch"]
end

```

```

TA -->|OTLP: metrics, logs, traces| P1
PS -->|OTLP: metrics, logs, traces| P1

```

```

P5 -->|logs| KAFKA["Kafka<br/>log buffer topic"]
KAFKA --> GW["OTel Collector — gateway"]
GW --> LOKI["Loki"]

```

```

P5 -->|traces| TEMPO["Tempo<br/>+ metrics generator"]
P5 -->|metrics| PROM["Prometheus"]

```

TEMPO -->|service graph + RED metrics| PROM

EOD -->|push| PGW["Pushgateway"]

PROM -->|scrape| PGW

subgraph INFRA["Infrastructure"]

NE["Platform host
node exporter"]

subgraph AWS["Simulated AWS"]

EC2API["Fake EC2 API"]

MD1["market-data-1
node exporter"]

MD2["market-data-2
node exporter"]

end

end

PROM -->|scrape| NE

PROM -->|discover by tags| EC2API

PROM -->|scrape| MD1

PROM -->|scrape| MD2

PROM -->|firing alerts| AM["Alertmanager"]

AM -->|email| MAIL["Local email inbox
on-call"]

GRAF["Grafana"]

GRAF -->|query| PROM

GRAF -->|query| LOKI

GRAF -->|query| TEMPO

9 Different Enterprise Challenges

How do I approach Explaining Enterprise Challenges & Solving them in practice?

- Problem TradeNova Reports
- Explain how the thing really works underneath - The mechanics
- Show the Problem - You shall see it's broken
- Explain how to fix & apply the fix
- Explore the Result

Scenarios

1. Unlabelled Data
2. Market-Open Spike
3. PII Compliance
4. Observability Cost
5. No Lost Logs
6. Silent Batch Failure
7. Scaling Fleet
8. On-Call Alerts
9. Slow Trades

Business Use Case 2

Lab Environment Setup

> Open the terminal in Administrator mode

> cd c:\demonstrations

> git clone <https://github.com/iomegak12/otlp-day2-platform>

> cd otlp-day2-platform

> Make sure that docker desktop is up and running

> docker --version
(2 dashes)

> In the docker desktop -> containers -> select tradenova -> delete the complete stack

```
> docker compose --profile tools --profile spike pull  
--ignore-buildable
```

> docker compose --profile tools build
(2 dashes)

> docker compose up -d

> docker compose ps

> open the docker desktop, and verify tradenova-day2 is up and running - all containers

Smoke Testing

> In vs code -> terminal -> bash

> curl localhost:8080/health

> curl localhost:8000/health

> Verify LoadGen is flowing

 > docker compose logs --tail 3 loadgen
(2 dashes)

> Verify Traces

> browser -> <http://localhost:3000> -> Grafana Home -> Explore -> Select Tempo ->
 Search -> Service = trade-api -> Run Query

> Verify Metrics

> browser -> <http://localhost:9090>

> Search -> type http_server_request_duration_seconds_count -> Execute Query

(both services should be visible - portfolio service and trade api)

> Verify Logs

> browser -> Grafana -> Explore -> select Loki ->

 Select service_name = trade_api -> run query

> Log lines should be having both INFO & DEBUG

> INFO should be including all sensitive business data

> Gateway and Kafka Checks

> docker compose logs otel-gateway

> No errors should be present

> docker compose logs kafka-init

> it should display tradenova-logs topic is created

> Verify Fake AWS

- > docker compose run --rm fleet status
- > 3 running market data instances should be visible
- > Verify Grafana Dashboards
- > browser -> Grafana -> Dashboards -> TradeNova
- > 4 Dashboards should be visible
- > Verify Mailpit
- > browser -> <http://localhost:8025>
- > Verify Batch Job does not send metrics / logs
- > docker compose logs --tail 5 eod-reconciliation

Problem #1: Unlabelled Data

Goal: Add Environment, Region, Cost Centre, Host, Owning Team to all Telemetry centrally at the agent, without modifying at source code | application level

Show the Problem:

-> Prometheus -> Query: target_info

You shall see only essential labels such as job, instance and version | SDK details, NOT meaningful business | customized information

> Grafana -> Explore -> Loki -> Label Browser

You shall only see OOB labels

> Grafana -> Explore -> Tempo -> TraceQL

`{resource.tradenova.name = "trading-core"}`

No results

Apply the Fix:

Refer the file scenarios\01-unlabelled-data\after\collector\agent\config.yaml and see what changes are to be brought to the OTEL collector agent

> terminal window -> git bash

> bash scripts/apply-scenario.sh 1

> wait for 20-30 seconds

> Verify the Solution

> Prometheus -> target_info -> Execute Query

Now you should be able to additional labels that we have introduced in the agent

> Grafana -> Loki -> Label Browser

> Grafana -> Loki -> Kick start your query -> select service_name = trade-api

> Grafana -> Loki -> Kick start your query -> select tradenova_team = trading-core

> Grafana -> Tempo -> TraceQL

> { resource.tradenova.team = "trading-core"} now returns traces

TIP to know whether Loki is working

> grafana -> loki -> switch to the code mode

{ service_name = "trade-api" } | tradenova_team = "trading-core"

Problem #2 - Market Open Spike

Goal: Show the agent being killed by a Traffic Spike, Then Protect with Memory Limiter and Batch

Show the Problem

> Browser -> Grafana -> TradeNova -> Platform Overview Dashboard

> Refer the Collectors Panel -> Collector Memory, Data Refused by the Agent and Exporter Queue Size

> VS Code -> Terminal window ->

> docker stats tradenova-day2-otel-agent-1

> Start the spike

> Open another terminal window (click + bash type)

> bash ./scenarios/02-market-open-spike/[trigger-spike.sh](#)

How do you know the spike?

> refer the grafana dashboard

> you should be able to see the memory limit goes high in the otel-agent-1 window

Apply the Fix:

Open the file scenarios\02-market-open-spike\after\collector\agent\config.yaml and refer the memory_limiter and batch sections

> vs code -> terminal window -> + Git bash

> bash ./scripts/[apply-scenario.sh](#) 2

> vs code -> terminal window -> + Git bash

> bash ./scenarios/02-market-open-spike/[trigger-spike.sh](#)

You should be seeing the dashboard - collectors memory | refusal rate is under control and otel agent memory is ALSO stable.

Problem #3: PII Compliance

Goal: Mask account numbers, email addresses and card numbers at the agent level, before they reach any back-end

How does it work?

- At the OTel collector agent level, we can apply transform processor with PII sections. And for logs, we can use replace pattern by just using regular expression to match.

Show the Problem:

> Grafana -> Explore -> Loki

Run the query

```
{ service_name = "trade-api" }
```

You should be able to see logs having full PII information including customer email, account number and so on

> Grafana -> Explore -> Tempo -> run the query

```
{ resource.service.name = "trade-api" }
```

Open the trace and see email and account number are part of the span.

Apply the Fix:

Open the file scenarios\03-pii-compliance\after\collector\agent\config.yaml

And identify the section transform/pii

Vs code -> terminal window -> + git bash

> bash ./scripts/[apply-scenario.sh](#) 3

Wait for 30 seconds ...

Verify the solution

> Grafana -> Explore -> Loki -> search for { service_name = "trade-api" }

> run the query

> now you should be able to see logs redacting Email and Account Numbers

> Grafana -> Explore -> Tempo -> run the query

```
{ resource.service.name = "trade-api" }
```

> You should now see trades not having the span including Email | account number is masked.

Problem #4: Observability Cost

Goal: Drop Health Check Traces, DEBUG logs and Cardinality with Metrics (The Per-Account Metric Label shall increase the number of time-series in Prometheus)

Show the problem:

> Grafana -> Dashboards -> Platform Overview -> Scroll down to Scenario 4

It shows hundreds | thousands of series, one per account

NOTE: EXPENSIVE in Storage, Querying and Visualization

> Grafana -> Explore -> Temp

Run the query { span.url.path = "/health" }

For every service, there are two health checks

> Grafana -> Explore -> Loki

Run the query { service_name = "trade-api" } | detected_level = "debug"

DEBUG lines are logged for every request

4. Prometheus run the following query

> count by (account_id) (tradenova_trades_placed_total) execute the query

You should see hundreds | thousands of entries

How do you fix this?

Open the file scenarios\04-observability-cost\after\collector\agent\config.yaml and navigate to the section filter/noise and metrics_transform

- You should see how we ignore health endpoints at traces | logs
- Reduce the number of time series per account

> vs code -> terminal window + git bash

> bash ./scripts/[apply-scenario.sh](#) 4

How do you verify?

> Grafana -> Dashboards -> Platform Overview -> Scroll down to Scenario 4

It shows only minimal number 5 or 10 ...

> Prometheus

Execute the following queries

> `sum by (symbol) (rate(tradenova_trades_placed_total{account_id=""}[1m]))`

> `count(tradenova_trades_placed_total{account_id=""})`

You should see the number 10 or minimal

> Grafana -> Loki->

Service name = trade-api

Run query -> you should now see NO DEBUG entries, only INFO entries.

> Grafana -> Tempo ->

Service name = trade-api

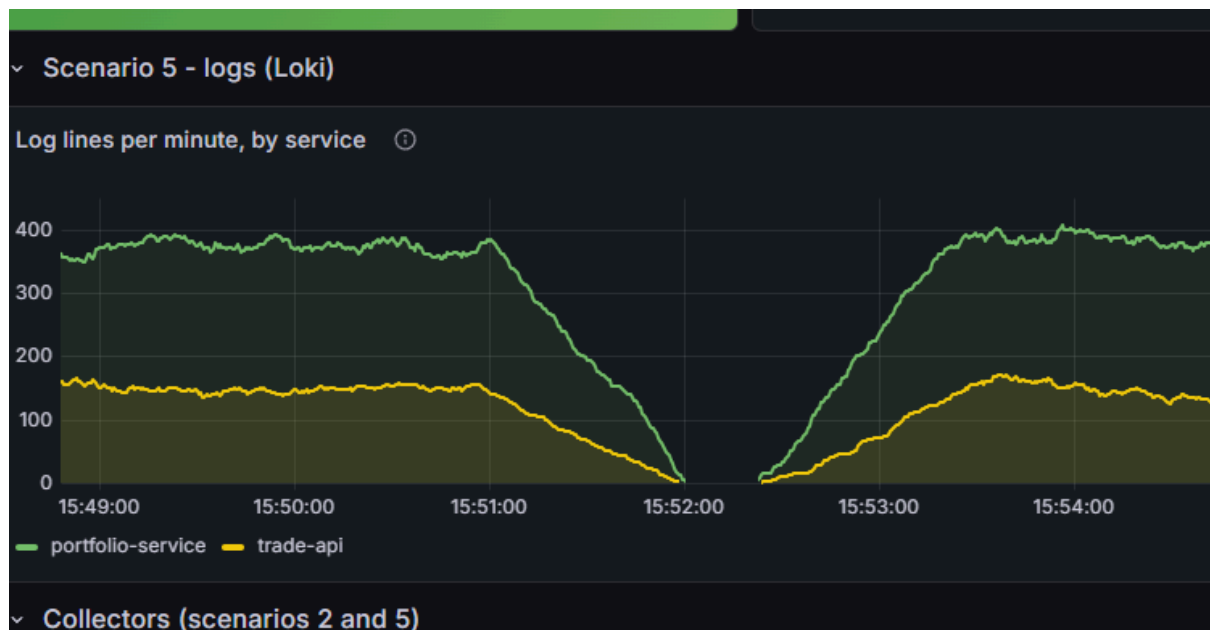
Run query -> you should now see No HEALTH CHECK endpoint traces

Problem #5: No Lost Logs

Goal: Prove that a Loki outage loses logs today, and bring Kafka broker between the agent and Loki and prove that nothing is lost

Show the Problem:

> Grafana -> Dashboards -> Platform Overview -> Scroll down to Section 5



Log lines per minute, by service - change the time to last 15 minutes

> terminal window -> + git bash

> bash ./scenarios/05-no-lost-logs/[loki-outage.sh](#) 120

> new terminal window -> + git bash

> docker compose logs --since 2m otel-agent | grep -i -E "dropping | failed"

Now, in the Grafana Dashboard, you should see a gap between two different log lines

Apply the fix

> open the file scenarios\05-no-lost-logs\after\collector\agent\config.yaml and locate the section kafka

> and see how Kafka broker is configured and connected

> Open the gateway YAML file collector\gateway\config.yaml, and see how Kafka broker is configured and connected

> terminal window -> + git bash

> MSYS_NO_PATHCONV=1 bash ./scenarios/05-no-lost-logs/[kafka-log.sh](#)

The lag column should be growing

> terminal window > + git bash

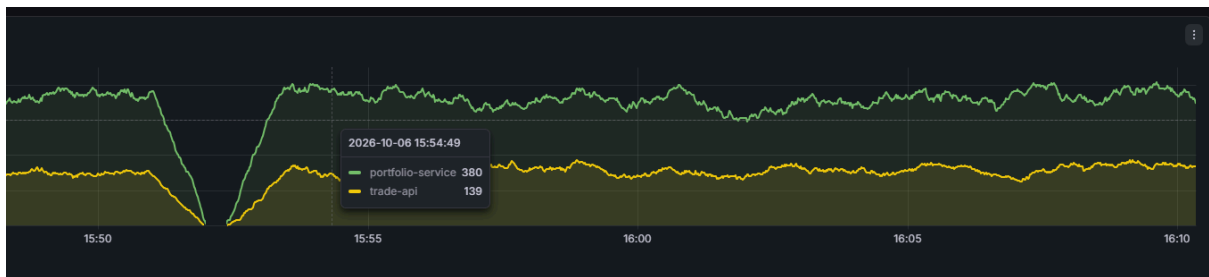
> bash ./scenarios/05-no-lost-logs/[loki-outage.sh](#) 120

> Refer the Kafka Lag, it should be growing

> terminal window > + git bash

> docker compose logs --since 1m otel-gateway

> here, you should see it's trying to connect LOKI, and once the Loki becomes available, it would automatically read from Kafka Broker and send those logs without any gaps



Problem #6: Silent Batch Failure

Goal: Make the short-lived jobs such as EOD-Reconciliation visible through the Push Gateway

Show the Problem:

> docker compose logs --tail 20 eod_reconciliation

Wait for a few minutes

> Prometheus -> search for eod_

> execute the query, you should see nothing

> Open the Push Gateway UI (<http://localhost:9091>)

> Refresh the Metrics, You should see nothing

Apply the fix

> open the file scenarios\06-silent-batch-failure\after\.env, and change the environment variable `EOD_PUSH_ENABLED` to `true`

> Open the VS code terminal window -> + git bash

> bash ./scripts/[apply-scenario.sh](#) 6

The job restarts and runs immediately.

How do you verify the solution?

> browser -> push gateway UI -> refresh the metrics

> prometheus -> `eod_job_success` -> execute the query

> Grafana -> Dashboards -> Batch Jobs -> you should now see the status as SUCCEEDED

If you want to experiment, always failure ...

> terminal window > + git bash ->

> bash ./scenarios/06-silent-batch-failure/failure-mode.sh random

> to revert back to Normal

> bash ./scenarios/06-silent-batch-failure/failure-mode.sh never

Problem #7: Scale Fleet (Service Discovery)

Goal: replace the hand-maintained market-data list of servers to scrape to automatically discovered using Service Discovery feature

Show the Problem:

> terminal window -> `bash ./scenarios/07-scaling-fleet/scale-fleet.sh status`

> Prometheus -> Status -> Targets

You should see market-data job has ONLY ONE market-data-1 instance which is manually configured

> Grafana -> Dashboards -> Hosts & Fleet

Discovered Instances is 1

How do you apply the fix?

Open the file `scenarios\07-scaling-fleet\after\infra\prometheus\prometheus.yml`, and refer the job name market data with EC2 service discovery configuration.

> terminal window -> `+ git bash > bash ./scripts/apply-scenario.sh 7`

Within 30 seconds, you should be able to see the changes

How do you verify the solution?

> refer the Prometheus -> Status -> Targets

> refer the Grafana Dashboard -> Host & Fleet -> Number of instances changed to 3

> terminal window -> `bash ./scenarios/07-scaling-fleet/scale-fleet.sh 5`

Within 30 seconds, you should be able to see the changes

> refer the Prometheus -> Status -> Targets

> refer the Grafana Dashboard -> Host & Fleet -> Number of instances changed to 3

Problem #8: On-Call Alerts

Goal: Show that failures wake nobody, and then load the alert rules and watch emails arrive for errors and resolution

Show the problem:

> vs code terminal window -> + git bash -> bash ./scenarios/08-on-call-alerts/[break-things.sh](#)

This script brings the market data instance 2 down, which is dependent by batch job and trade-api requests

> Show the damage

- > Grafana -> Dashboards -> Hosts & Fleet
- > Grafana -> Dashboards -> Platform Overview (5xx errors)
- > Grafana -> Dashboards -> Batch Jobs FAILED

> Prometheus -> Alerts -> No Alerting Rules

How do you apply the fix?

Open the file scenarios\08-on-call-alerts\after\infra\prometheus\prometheus.yml, which has alert manager configuration.

Open the file infra\alertmanager\alertmanager.yml, to understand how alerts are handled | sent to email servers

Open the folder infra\prometheus\rules, which has YAML files for individual rules to be evaluated on a scheduled basis and generate alerts

> terminal window > + git bash > bash ./scripts/[apply-scenario.sh](#) 8

> another terminal window > + git bash > bash ./scenarios/08-on-call-alerts/[break-things.sh](#)

How do you verify?

> prometheus alerts are generated

> prometheus status -> target health - one of the instances is DOWN

> Alert Manager UI -> now generates alerts

> Emails are received and verify at Mailpit

> Grafana Dashboards

If you want to send recovery solution, > another terminal window > + git bash > bash
./scenarios/08-on-call-alerts/[heal.sh](#)

Problem #9: Slow Trades

Goal: Find out which hop is slow, then switch to Tempo's METRIC generator for a service graph and RED metrics

Show the problem:

> make the trade-api slow

> terminal window -> + git bash > bash ./scenarios/09-slow-trades/inject-latency.sh 800

> grafana -> dashboards -> platform overview -> p95 latency is gradually increasing the time

> grafana -> explore -> tempo -> service graph tab -> EMPTY

> grafana -> explore -> Tempo -> TraceQL

{ [resource.service.name](#) = "portfolio-service" && duration > 500ms }

Run the query

> View Traces -> span -> inside, you can see trade-api is consuming more time

Apply the fix:

Open the file scenarios\09-slow-trades\after\infra\tempo\tempo.yaml, and refer the section overrides, which provides the configuration for metrics generator to service graph and span related metrics

> terminal window > + git bash > bash ./scripts/[apply-scenario.sh](#) 9

Wait for 1 or 2 minutes

How do you verify the solution?

> grafana -> explore -> tempo -> service graph

Now you should be able to visualize how services communicate each other, and their edge level metrics

> grafana -> dashboards -> tradenova -> services RED

> you should be able to see all metrics are populated and showing

> TraceQL example

```
{ resource.service.name = "portfolio-service" && duration > 500ms }
```

Run query -> view traces & spans -> Navigate to Logs also

> Grafana -> loki -> search for service name to portfolio-service and any one of the log lines should allow you to navigate to Tempo traces ...

> terminal window > + git bash >

> terminal window -> + git bash > bash ./scenarios/09-slow-trades/inject-latency.sh 0

Day 3 Agenda

- Day 1 & Day 2 Summary
- Business Use Case 3 - Advanced
 - K8S Deployment
 - Auto-Instrumentation with the K8S OTEL Operator
 - Agent, Gateway and Collectors - Scaling with K8S
 - Queuing & PVC (Persistent Volume Claims) - Persisted Storage
 - Monitoring - OTEL Collector Agents | Gateways & Back-End
 - Custom Dashboards
 - PromQL | TraceQL - Complex Queries
 - Alerting
 - Security & Governance
- Move Forward with AI Automation
 - AIOps
- Wrap with QA, Feedback | Contact | Materials

Business Use Case 3 - Advanced

Today's Theme - Production, Scale and Trust

4 Important - Production Grade | Enterprise Challenges

- Auto Instrumentation with K8S Operator
- Agent, Collector and Gateway - Kafka Brokers - Scalability
 - Durability with Queuing | Persistent Storage
- Monitoring OTEL Stack (Collectors, Agents, Gateways, Prometheus | Grafana | Loki | Tempo)
- Enterprise Security & Governance

Mermaid Contents for Business Use Case 3 Architecture

flowchart LR

```
subgraph APPS["namespace tradenova: no OpenTelemetry in any image"]
    direction TB
    LG["loadgen<br/>about 5 requests/s"]
    PS["portfolio-service<br/>Python x2, team = wealth"]
    TA["trade-api<br/>Java x2, team = trading"]
    NOTIF["notification-service<br/>Node.js x2, team = trading<br/>deployed live in Lab 1"]
    EOD["eod-reconciliation<br/>Python CronJob, every 2 min"]
    LG --> PS
    LG --> TA
    EOD --> PS
    PS --> TA
    TA --> NOTIF
end
```

```
subgraph AGENTS["observability: agent tier"]
    AGENT["Agent Collector<br/>DaemonSet, 1 per worker node<br/>memory_limiter, k8sattributes, batch<br/>disk queue on the node: Lab 2"]
end
```

```
subgraph KAFKA["namespace kafka"]
    K["Kafka, 3 brokers<br/>otlp-traces, otlp-metrics, otlp-logs<br/>6 partitions each"]
end
```

```
subgraph GATEWAYS["observability: gateway tier"]
    GW["Gateway Collector<br/>StatefulSet, 2 to 6 pods, HPA: Lab 1<br/>disk queue on a PVC: Lab 2<br/>allowlist + tenant routing: Lab 4"]
end
```

```
subgraph MON["namespace monitoring"]
    direction TB
    TEMPO["Tempo<br/>multi-tenant"]
    LOKI["Loki<br/>multi-tenant"]
end
```

```
PROM[("Prometheus<br/>remote write + scrape")]
GRAF["Grafana :30300<br/>one data source per tenant"]
end
```

```
APPS ==>|"OTLP: traces, metrics, logs<br/>mTLS + token from Lab 4"| AGENT
AGENT ==>|"one topic per signal<br/>TLS client cert from Lab 4"| K
K ==>|"consumer group otel-gateway<br/>partitions shared out"| GW
GW ==>|"traces"| TEMPO
GW ==>|"logs"| LOKI
GW ==>|"metrics, remote write"| PROM
GRAF -.-> TEMPO
GRAF -.-> LOKI
GRAF -.-> PROM
```

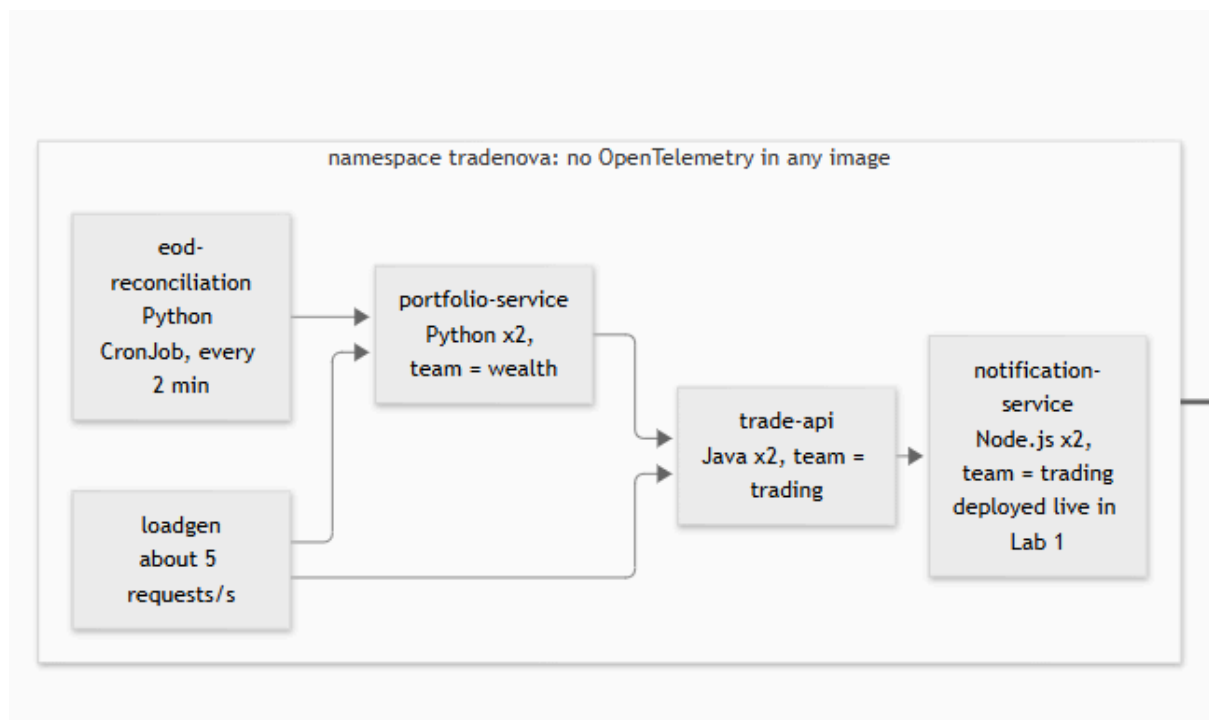
```
subgraph PLATFORM["Platform operators, installed with Helm"]
  direction LR
  OP["OpenTelemetry Operator<br/>injects agents into pods<br/>runs the Collector
CRs"]
  STZ["Strimzi<br/>brokers, topics, users"]
  CM["cert-manager<br/>lab CA and certificates"]
  SCRAPE["Prometheus also scrapes<br/>Collector metrics :8888<br/>Kafka consumer
lag<br/>kube-state-metrics"]
end
```

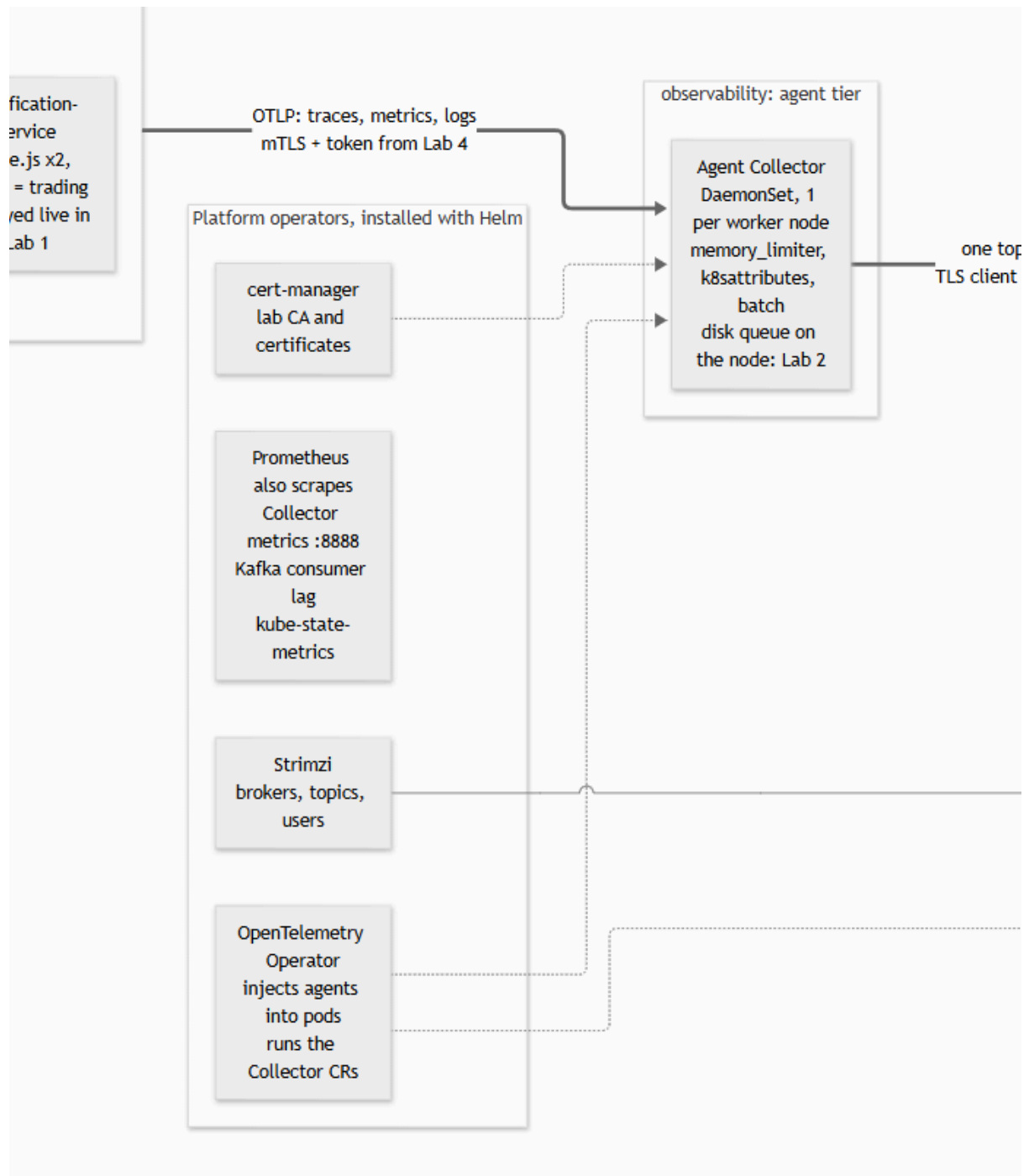
```
OP -.-> AGENT
OP -.-> GW
STZ -.-> K
CM -.-> AGENT
```

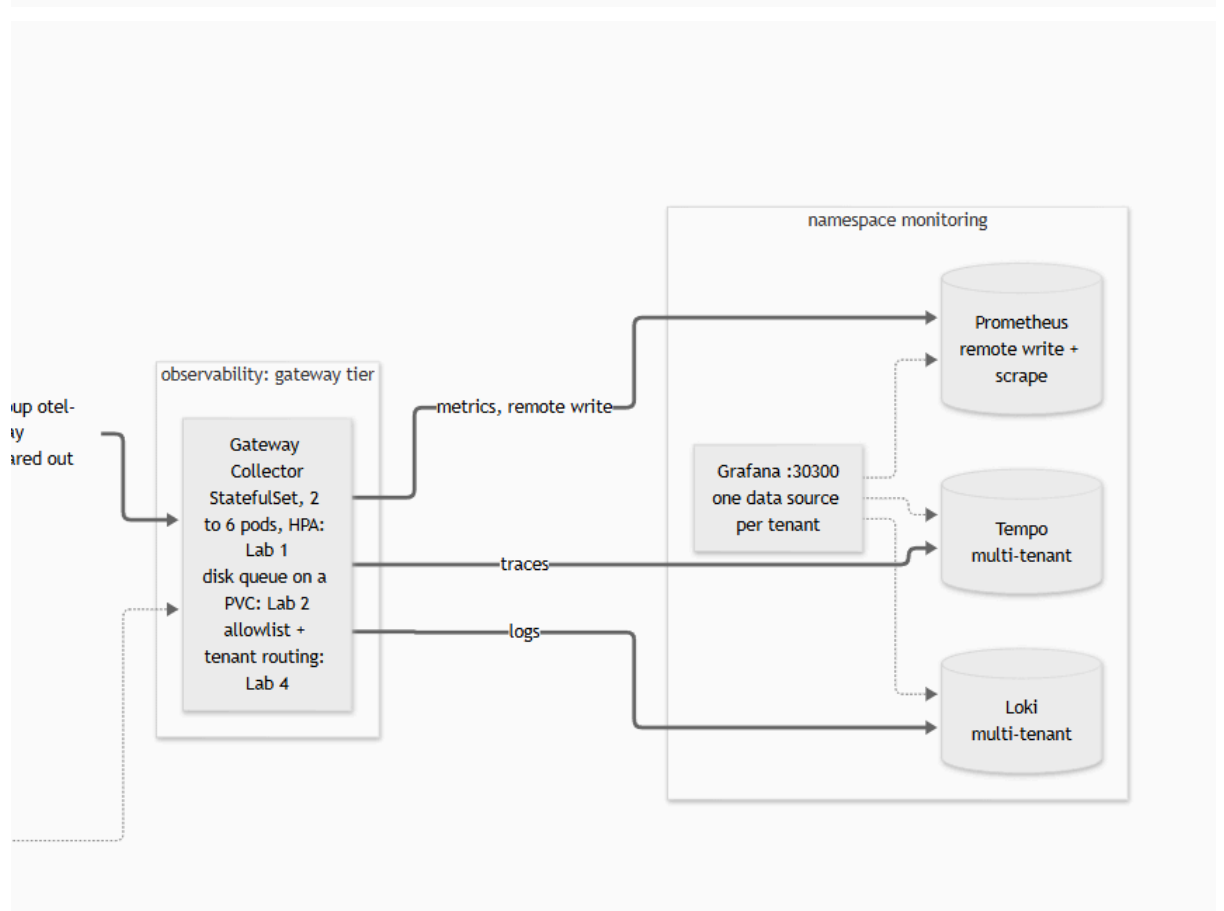
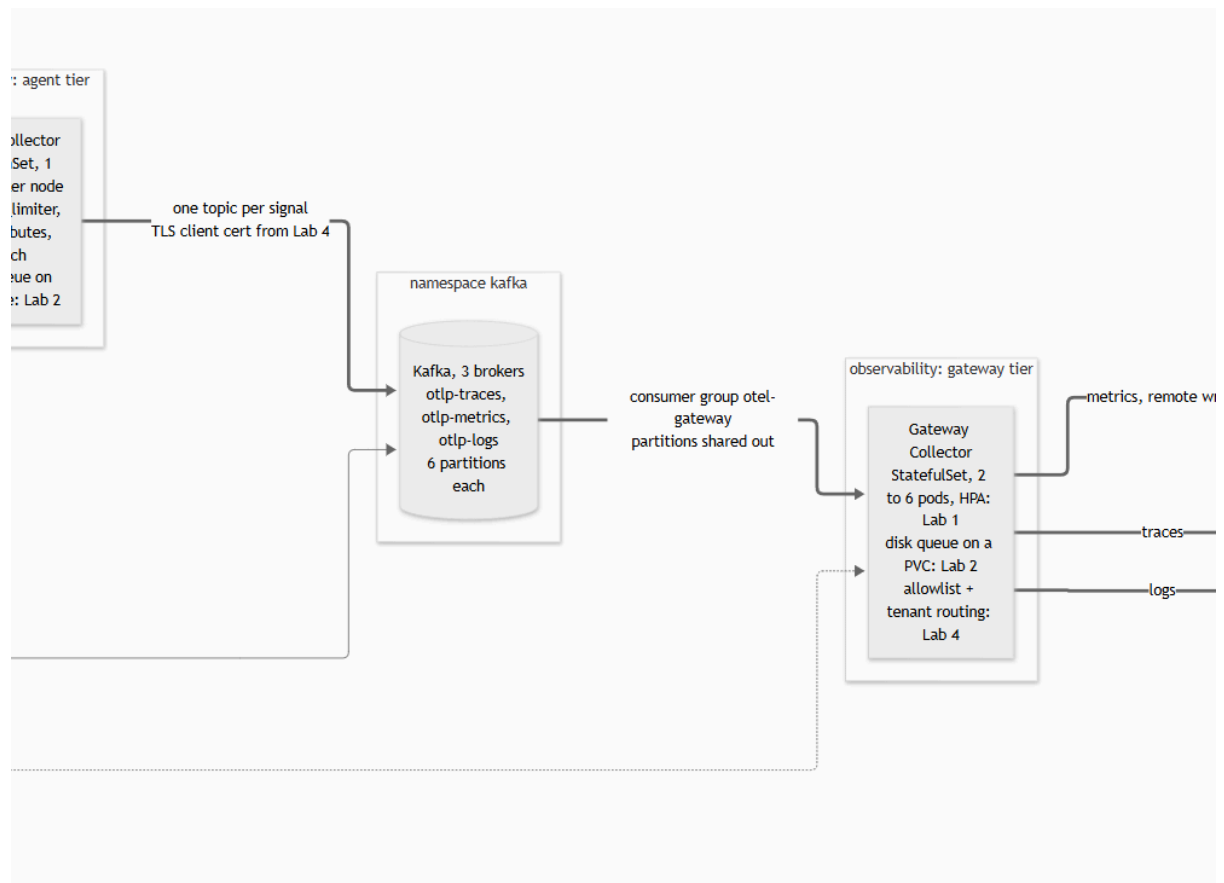
Applications send telemetry to an agent on their own node. The agents drop them into Kafka, a group of gateways read from Kafka Broker and deliver them to Tempo, Loki and Prometheus. Grafana sits on the top.

Five Hops

- Applications
- Agents
- Kafka
- Gateways
- Back-Ends







Ensure that the following tools and utilities are available

```
git --version
multipass --version

kubectl version
docker --version
helm version
```

In the terminal window administrator mode ->

> cd c:\demonstrations

> git clone <https://github.com/iomegak12/otlp-day3-platform>

> cd otlp-day3-platform

> code .

> in vs code -> open lab.env

> update the REGISTRY environment variable to your Docker Hub account

> Terminal -> Git Bash

> 00-cluster/[create-vms.sh](#)

> you shall be able to see 4 VMs are created (1 Control Plane and 3 Worker Nodes)

> 00-cluster/[install-k3s.sh](#)

> This shall provision K3S Server and 3 Workers

> This shall also automatically copy the kube configuration file into .kubeconfig folder

> kubectl get nodes

> kubectl get pods --all-namespaces
(2 dashes)

> Build and Push Images

> scripts/[build-and-push.sh](#)

> Browser -> hub.docker.com -> login with your Docker Hub Id -> Repositories

> you should see 5 repositories

> Install the platform

> helm/[install-platform.sh](#)

It installs in order - Namespaces, Certificate Manager, OTEL operator, Strimzi Kafka Operator and OTEL Back-Ends, Instrumentation and Application

> Smoke Testing

> scripts/[status.sh](#)

> If you want to see URLs to access Prometheus | Grafana,

> scripts/[urls.sh](#)

> agents and gateways in kubectl

```
kubectl get pods -n observability -o wide
```

> Verify Injection

```
> labs/lab1-agent-gateway-scaling/show-injection.sh  
trade-api
```

```
> Kafka Lag
```

```
> scripts/kafka-lag.sh
```

> Grafana -> Explore -> Select the source -> Tempo (Shared) -> search

You should see all service names in Shared Only

But if you switch to Tempo (Wealth) or Tempo (Trading) - you see no records

> Grafana -> Explore -> Select the source -> Loki (Shared) -> search

You should see all service names in Shared Only

But if you switch to Loki (Wealth) or Loki (Trading) - you see no records

> Prometheus, verify the following metrics

1. `Http_server_request_duration_seconds_count`

2. Otelcol_receiver_accepted_spans
3. Otelcol_receiver_refused_spans
4. kafka_consumergroup_lag

Prometheus -> Alerts (By default few alerts have been configured)

Lab 1: Auto Instrumentation with the OTEL Operator

The real problem is OWNERSHIP.

Two pieces make this work.

- Custom Resource - Instrumentation
- Mutating Webhook
 - This is a Kubernetes Hook that lets the operator edit the pod before it starts

Common Challenges

- The annotation has to be on the pod template
- The webhook only acts when a pod is created
- Language Images are preferred. Otherwise, deployment should add additional metadata. If you use any non-language images, such as alpine with a different C library may break

Goal: Show that no image contains OpenTelemetry, show what the Operator injects, then deploy a new Node JS service live.

How did we fix this?

- Open the file k8s\instrumentation\instrumentation.yaml
- Understand how does endpoint, propagators and environment variables are injected to pods
- Open the file k8s\apps\trade-api.yaml, and line #18 highlights that how pod template defines an annotation metadata to describe the language
- labs/lab1-agent-gateway-scaling/show-injection.sh portfolio-service
- labs/lab1-agent-gateway-scaling/show-injection.sh trade-api
- Understand how OTEL operator injected the instrumentation specific details to pods
- Deploy the Notification Service
- labs/lab1-agent-gateway-scaling/deploy-notification-service.sh

Open the browser -> Grafana -> Loki (shared) -> Label Browser

You should see K8S specific attributes

You should see Notification service specific logs

You should see trade-api is properly calling notification service

By default, console.log outputs are NOT redirected to Loki ...

labs/lab1-agent-gateway-scaling/[lab1a-resolution.sh](#)

This shall configure OTEL agents to redirect stdout file to FileLogReceiver to OTEL collector.

Now, you should be able to see Notification Service log also

Lab 1B - Agent + Gateway - Scalability

HPA - Horizontal Pod Autoscaler

The HPA is a small control loop inside K8S. Every 15 seconds, it asks the metrics server: How much CPU are the gateways using, compared to what they requested? Our gateways request a tenth of a CPU, and our target is 60%, when the average goes above that, HPA adds pods.

Goal: One gateway falls behind during a burst, the HPA scales the gateways to SIX and Kafka spreads the partitions.

What's the problem?

Open a terminal -> bash ->

> labs\lab1-agent-gateway-scaling\[watch-scaling.sh](#)

By default it's NO AUTOSCALER.

Start the burst

> labs\lab1-agent-gateway-scaling\[burst-load.sh](#)

Within 1-2 minutes, you should be able to see log on traces go up ...

> prometheus -> query

> sum by (topic) (kafka_consumerlag{consumer="otel-gateway"})

Switch to graph mode

> stop the burst

> labs\lab1-agent-gateway-scaling\[stop-burst.sh](#)

How do you fix this?

Open the file labs\lab1-agent-gateway-scaling\after\k8s\collectors\gateway.yaml, and scroll down to autoscaler section ...

> terminal window -> scripts/[apply-lab.sh](#) 1

Start the burst

Now, you should be able to see number of gateway pod instances grow and kafka lag is still properly maintained.

Lab 2: Persistent Queues

Goal: A backend outage and a gateway crash loses data with in-memory queues, with the configuration of PVC based disk-queues and message_marking, NOTHING is LOST (Kafka also keeps them and Gateway also retains in the disk, when the gateway pod recovered | restarted, K8S attaches the disk which has the queue data)

See the problem:

> terminal window -> git bash >

> labs\lab2-persistent-queues\[queue-status.sh](#) -watch
(2 dashes)

By default queue is 0, i.e. all messages are read and sent to the back-end

> another terminal window ->

> labs\lab2-persistent-queues\[backend-outage.sh](#) start

Over the next 1-2 minutes, tempo and loki queues are growing on every gateway

Prometheus is UP and remote write is 0 ONLY

So far nothing is lost, it's waiting in MEMORY

After about 2 minutes, the maintenance on node where LOKI | TEMPO ... kill all gateways

> labs\lab2-persistent-queues\[kill-gateways.sh](#)

Queue size before the kill and after the restart ... by default it's set to 0 (what was queued is GONE)

> labs\lab2-persistent-queues\[backend-outage.sh](#) stop

Wait for 1-2 minutes, Loki and Tempo should be having a hole in the graph and logs collection.

How do you fix this?

Open the file labs\lab2-persistent-queues\after\k8s\collectors\gateway.yaml, and refer the extensions section for file storage and receivers along with exporters how they're related and importantly message marketing is set to AFTER TRUE

Apply the fix

> scripts/[apply-lab.sh](#) 2

Run the same outage again ... This time, you should NOT be missing logs and traces

```
> labs\lab2-persistent-queues\backend-outage.sh start
```

```
> labs\lab2-persistent-queues\kill-gateways.sh
```

```
> labs\lab2-persistent-queues\backend-outage.sh stop
```

You should be able to see the same number of queue batches before the kill and after the kill.

Lab 3: Collector Monitoring - Health Dashboard Customized + Broken Pipelines

Problem Statement: If one exporter breaks, the traces stop, but it may so happen that metrics may be flowing. Your dashboards may still look alive. If only the logs topic stops being read by gateway, everything looks perfect. Most of the times, the worst pipeline failures are the partial ones. They're the silent killers.

Our objective is to NOT to query "is pod running?" for gateways / kafka brokers / collector agents?

Commonly asked questions in production

- Is Data coming in?
- Is it moving through the middle?
- Is Data going out?
- What's waiting?

Important Metrics to analyze:

- Otelcol_exporter_queue_capacity
- Otelcol_exporter_queue_size
- Otelcol_receiver_accepted_spans
- Otelcol_receiver_refused_spans
- Otelcol_exporter_sent_log_records
- Otelcol_exporter_sent_metric_points
- otelcol_exporter_sent_spans

Lab 4: Security and Governance

4 Enterprise Challenges

- Anyone in the cluster can send the data to our agents
- The data travels in Plain Text (HTTP)
- We send every attribute we get
- Everything goes to a shared bucket

Apps to Agent - mTLS + Token

Agent to Kafka - mTLS + Kafka User

At the Gateway - Attribute Allowlist + Tenant Routing based on Team

Goal: Lock down every hop, drop unapproved attributes and route data to per-team tenants.

Show the Problem?

> terminal -> git bash

> labs\lab4-security\[test-agent-security.sh](#)

By default only HTTP is allowed - No HTTPS, No Certification, No Token based Authentication

> labs\lab4-security\[peek-kafka.sh](#)

Anybody who has access to the Kafka Broker, can read messages

> Grafana -> Loki (Trading) | Loki (Wealth) - Show no data

> Grafana -> Tempo (Trading) | Tempo (Wealth) - Show No Data

> If you see any logs from Loki | Traces from Tempo, you may be able to see Email Address
| Account Number ...

(We want to take a call - Only certain attributes are allowed, NOT everything by applications)

Apply the fix:

> kubectl get certificates -A

> kubectl get kafkauser -n observability

> scripts/[apply-lab.sh](#) 4

Open the file labs\lab4-security\after\k8s\instrumentation\instrumentation.yaml, understand how exporter and environment variables are configured

Open the file labs\lab4-security\after\k8s\collectors\gateway.yaml, and see how mTLS certificate is configured

Open the file labs\lab4-security\after\k8s\collectors\agent.yaml, and see how mTLS, authentication, allowlist and routing configured

How do you verify?

> labs\lab4-security\test-agent-security.sh

> labs\lab4-security\test-kafka-tls.sh

> labs\lab4-security\tenant-check.sh

In Grafana -> you should now be able to view Loki (Wealth) | Loki (Trading) | Tempo (Wealth) | Tempo (Trading) data sources. Now the shared data source won't show anything.